julien.seinturier@univ-tln.fr

CV / Homogeneous Coordinates [Python]

This practical work is designed to illustrate the use of Homogeneous Coordinates for representing 3D points, transformations and calculus.

Prerequisite

MatPlotLib configuration

We're using the Python language and the MatPlotLib library. The MatPlotLib library can be installed with the command:

```
pip install matplotlib
```

or for conda installation:

```
conda install -c conda-forge matplotlib
```

Using MatPlotLib within a Python program need to add the import:

```
import matplotlib.pyplot as plt
```

The MatPlotLib library displays geometric rendering in an independent interactive window. Depending on the Python environment, this window may be rendered as a frozen image, preventing user interaction. To correct this problem, here are solutions according to the underlying Python environment.

Jupyter Notebook

Execute following code within the Jupyter Notebook:

```
%matplotlib qt
```

PyCharm

Go to `Settings / Tool / Python Plot` and uncheck the option `Show plots in tool windows`.

Spyder

Go to `Tools / Preferences / IPython console / Graphics / Backend:Inline` and change "`Inline`" to "`Automatic`". Click OK button and restart the IDE.

MatPlotLib Drawing functions

MatPlotLib enable to display various primitives and shapes. In this work we focus on drawing points and lines.

Point

The drawing of a point with $(x,\ y,\ z)$ coordinates is made by the code:

```
plt.plot(x, y, z, marker='m', color='c')
```

where:

- `m` is the marker (shape) to use for displaying the point ('`o`' for round, '`+`' for right cross, '`x`' for cross)
- `c` is the point color ('red', 'greed', 'blue', ...)

Line

The drawing of a line between two points (x_1, y_1, z_1) and (x_2, y_2, z_2) is made by the code:

```
plt.plot([x1 , x2],[ y1, y2],[ z1, z2], color='c', lines
```

where:

- `c` is the line color ('`red`', '`greed`', '`blue`', ...)
- `s` is the line style ('`solid`', '`dashed`', '`dashdot`' or '`dotted`')

Matrix calculus

This work relies on Numpy for the vector and matrix representation. The Numpy library can be integrated within Python program with the import:

```
import numpy as np
```

Points and vectors

Points can be represented as Numpy array. Creating a point $p = (x, y, z)$ can be done using the python instruction:

```
p = np.array([x, y, z])
```

Simple operations

Let $p = (x, y, z)$ and $q = (t, u, v)$ two points, the result of adding, subtracting, or multiplying p and q are given respectively by:

```
p = np.array([x, y, z]) # Create and initialize p

q = np.array([t, u, v]) # Create and initialize q

plus = p + q            # Sum = (p[0]+q[0], p[1]+q[1], p

diff = p - q            # Difference = (p[0]-q[0], p[1]-

mult = p * q            # Multiplication = (p[0]*q[0], p
```

Dot and cross product

Let $p = (p_x, p_y, p_z)$ and $q = (q_x, q_y, q_z)$ two vectors, the result of dot product ($\cdot$) or cross product ($\times$) are given respectively by:

$$p \cdot q = \begin{pmatrix} p_x \\ p_y \\ p_z \end{pmatrix} \cdot \begin{pmatrix} q_x \\ q_y \\ q_z \end{pmatrix} = p_x qx + p_y q_y + p_z q_z$$

and

$$p \times q = \begin{pmatrix} p_x \\ p_y \\ p_z \end{pmatrix} \times \begin{pmatrix} q_x \\ q_y \\ q_z \end{pmatrix} = \begin{pmatrix} p_y q_z - p_z q_y \\ p_z q_x - p_x q_z \\ p_x q_y - p_y q_x \end{pmatrix}$$

The corresponding Python implementation is:

```
p = np.array([x, y, z])
q = np.array([t, u, v])

dot = p.dot(q)          # Dot = (p[0]*q[0] + p[1]*q[1] +

cross = np.cross(p, q) # Cross = (p[1]*q[2] - p[2]*q[1],
```

Exercise 1.

Create a python program that define two points $a = (1, 0, 0)$ *and* $b = (0, 1, 0)$ *and that computes display:* $a + b$, $a - b$, $a * b$, $a \cdot b$, *and* $a \times b$

Matrix

Matrix can be represented as Numpy arrays. Let the matrix M with l lines and c columns defined such as:

$$M = \begin{bmatrix} m_{00} & \cdots & m_{0j} & \cdots & m_{0c} \\ \vdots & \ddots & \vdots & & \vdots \\ m_{i0} & \cdots & m_{ij} & \cdots & m_{ic} \\ \vdots & & \vdots & \ddots & \vdots \\ m_{l0} & \cdots & m_{lj} & \cdots & m_{lc} \end{bmatrix}$$

The representation of M within a python program is given by:

```
M = np.array(((m00, …, m0c), …, (mi0, …, mic), …, (ml0,
```

Exercice 2.

Create a python program that define and display the matrix:

$$M = \begin{bmatrix} 0 & 0 & 1 \\ 0 & 1 & 0 \\ 0 & 1 & 1 \end{bmatrix}$$

Matrix / vector multiplication

Let M be a matrix made of l lines and c columns and let V be a vector of dimension c represented as a column matrix. We have:

$$M = \begin{bmatrix} m_{00} & \cdots & m_{0j} & \cdots & m_{0c} \\ \vdots & \ddots & \vdots & & \vdots \\ m_{i0} & \cdots & m_{ij} & \cdots & m_{ic} \\ \vdots & & \vdots & \ddots & \vdots \\ m_{l0} & \cdots & m_{lj} & \cdots & m_{lc} \end{bmatrix} and\, V = \begin{bmatrix} v_0 \\ \vdots \\ v_j \\ \vdots \\ v_c \end{bmatrix}$$

The result of the multiplication of V by M, denoted $R = MV$, is a vector where each element i is the result of the dot product $\cdot$ of the line i of M with V. More formally:

$$R = \begin{bmatrix} r_0 \\ \vdots \\ r_i \\ \vdots \\ r_l \end{bmatrix}, with\, r_i = \begin{bmatrix} m_{i0} \\ \vdots \\ m_{ij} \\ \vdots \\ m_{ic} \end{bmatrix} \cdot V = \sum_{j=0}^{c} m_{ij} v_j$$

The Python implementation of the matrix / vector multiplication relies on the dot product:

```
V = np.array([v0, ..., vj, ..., vc]) # A vector of c ele

M = np.array(((m00, ..., m0c), ..., (mi0, ..., mic), ...

R = M.dot(V) # R = MV
```

Exercise 3.

Write a python program that compute and display the matrix / vector product:

$$R = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 2 & 0 \\ 0 & 0 & 3 \end{bmatrix} \begin{bmatrix} 4 \\ 5 \\ 6 \end{bmatrix}$$

Matrix / matrix multiplication

Let M and K be matrices of size $l \times s$ and $s \times c$ respectively:

$$M = \begin{bmatrix} m_{00} & \cdots & m_{0t} & \cdots & m_{0s} \\ \vdots & \ddots & \vdots & & \vdots \\ m_{i0} & \cdots & m_{it} & \cdots & m_{is} \\ \vdots & & \vdots & \ddots & \vdots \\ m_{l0} & \cdots & m_{lt} & \cdots & m_{ls} \end{bmatrix} \text{ and } K = \begin{bmatrix} k_{00} & \cdots & k_{0j} & \cdots & k_{0c} \\ \vdots & \ddots & \vdots & & \vdots \\ k_{t0} & \cdots & k_{tj} & \cdots & k_{tc} \\ \vdots & & \vdots & \ddots & \vdots \\ k_{s0} & \cdots & k_{sj} & \cdots & k_{sc} \end{bmatrix}$$

The result of the multiplication of K by M, denoted $R = MK$, is a matrix of size $l \times c$ where each element r_{ij} is the result of the dot product $\cdot$ of the line i of M with column j of K. More formally:

$$R = \begin{bmatrix} r_{00} & \cdots & r_{0j} & \cdots & r_{0c} \\ \vdots & \ddots & \vdots & & \vdots \\ r_{i0} & \cdots & r_{ij} & \cdots & r_{ic} \\ \vdots & & \vdots & \ddots & \vdots \\ r_{l0} & \cdots & r_{lj} & \cdots & r_{lc} \end{bmatrix}, \text{ with } r_{ij} = \begin{bmatrix} m_{i0} \\ \vdots \\ m_{it} \\ \vdots \\ m_{is} \end{bmatrix} \cdot \begin{bmatrix} k_{0j} \\ \vdots \\ k_{tj} \\ \vdots \\ k_{sj} \end{bmatrix} = \sum_{t=0}^{s} m_{it} k_{tj}$$

The corresponding Python implementation is:

```
M = np.array(((m00, ..., m0c), ..., (mi0, ..., mic), ...

K = np.array(((k00, ..., k0c), ..., (kt0, ..., ktc), ...

R = M.dot(K) # R = MK
```

Exercise 4.

Write a python program that compute and display the matrix / matrix product:

$$R = \begin{bmatrix} 1 & 0 & 0 & 1 \\ 0 & 2 & 0 & 4 \\ 0 & 0 & 3 & 1 \end{bmatrix} \begin{bmatrix} 4 & -1 \\ 3 & 2 \\ 2 & -5 \\ 6 & 4 \end{bmatrix}$$

Transformations within homogeneous coordinates

We are now focusing on 3D point transformation implementation and display.

Homogeneous points and vectors

Let $P = (x, y, z)$ be a point expressed within Euclidean Space with Cartesian Coordinates. A representation of P within Homogeneous coordinates is the column matrix:

$$\mathrm{P} = \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix}$$

Let Q a vector expressed within Homogeneous coordinates such as:

$$\mathrm{Q} = \begin{bmatrix} t \\ u \\ v \\ \omega \end{bmatrix}$$

Q is represented within Euclidean Space as $Q = \left(\frac{t}{\omega}, \frac{u}{\omega}, \frac{v}{\omega}\right)$

Exercise 5.

Create a Python program named **`homogeneous.py`** *and implement a function* **`toHomogeneous(v: tuple) -> np.array`** *that takes in parameter a* `tuple v` *representing a vector expressed within homogeneous coordinates and that returns a* `np.array` *representing the equivalent vector within Euclidean coordinates. Test the program by converting the vector* $(1.0, 2.0, 3.0)$ *to Homogeneous Coordinates. The converted vector should be* $(1.0, 2.0, 3.0, 1.0)$.

Exercise 6.

Within **`homogeneous.py`**, *add a function* **`toEuclidean(v: tuple) -> np.array`** *that takes in parameter a* `tuple v` *representing a vector expressed within homogeneous coordinates and that returns a* `np.array` *representing the equivalent vector within Euclidean coordinates. Test the program by converting the vector* $(2.0, 4.0, 6.0, 2.0)$ *to Euclidean coordinates. The converted vector should be* $(1.0, 2.0, 3.0)$.

Translation

A translation is an application, denoted $T(\alpha,\ \beta,\ \gamma)(P)$ that can be represented within Homogeneous coordinates by the matrix:

$$\mathrm{T}(\alpha,\ \beta,\ \gamma)\ =\ \begin{bmatrix}1 & 0 & 0 & \alpha\\ 0 & 1 & 0 & \beta\\ 0 & 0 & 1 & \gamma\\ 0 & 0 & 0 & 1\end{bmatrix}$$

Let $P\ =\ (x, y, z)$ a 3D point, the translation of P along the vector $(\alpha,\ \beta,\ \gamma)$ can be represented within homogeneous coordinates as:

$$\mathrm{T}(\alpha,\ \beta,\ \gamma)(\mathrm{P})\ =\ \begin{bmatrix}1 & 0 & 0 & \alpha\\ 0 & 1 & 0 & \beta\\ 0 & 0 & 1 & \gamma\\ 0 & 0 & 0 & 1\end{bmatrix}\begin{bmatrix}x\\ y\\ z\\ 1\end{bmatrix}$$

Exercise 7.

Within `homogeneous.py`*, add a function* `translate_point_hc(point, alpha, beta, gamma)` *that takes in parameter the homogeneous representation of a point* $(x,\ y,\ z)$ *and that return the homogeneous vector resulting of the translation of* $(x,\ y,\ z)$ *along vector* $(\alpha,\ \beta,\ \gamma)$.

Test the function `translate_point_hc(point, alpha, beta, gamma)` *by displaying the point* $(4.0,\ 3.0,\ 2.0)$ *and by displaying the result of its translation along vector* $(0.0,\ 1.0,\ 1.0)$.

Tip: *Keep in mind to convert from euclidean to homogeneous coordinates and from homogeneous to euclidean coordinates when needed.*

Rotation

The rotation $R_x(\omega)$ around X axis by an angle ω is an application that can be represented by the matrix:

$$\mathrm{R}_x(\omega)\ =\ \begin{bmatrix}1 & 0 & 0 & 0\\ 0 & cos(\omega) & -sin(\omega) & 0\\ 0 & sin(\omega) & cos(\omega) & 0\\ 0 & 0 & 0 & 1\end{bmatrix}$$

Let $P\ =\ (x,\ y,\ z)$ a 3D point, the rotation $R_x(\omega)(P)$ can be

represented within homogeneous coordinates by:

$$\mathrm{R}_x(\omega)(\mathrm{P}) = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & cos(\omega) & -sin(\omega) & 0 \\ 0 & sin(\omega) & cos(\omega) & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix}$$

Exercise 8.

Within **`homogeneous.py`**, *add a function* **`rot_x_point(point, omega)`** *that takes in parameter an homegeneous vector that represents a* `point` $(x,\ y,\ z)$ *and returns the homogeneous vector that represents the rotated* `point` *around* X *axis by an angle* `omega` (expressed in radians).

Test the function **`rot_x_point(point, omega)`** *by displaying the point* $(4.0,\ 3.0,\ 2.0)$ *as a* ***black circle*** *and by displaying the result of its rotation around* X *axis by an angle of* $\frac{\pi}{6}$ as a **red circle**.

Tip: *Keep in mind to convert from euclidean to homogeneous coordinates and from homogeneous to euclidean coordinates when needed.*

The rotation $R_y(\phi)$ around Y axis by an angle ϕ is an application that can be represented by the matrix:

$$\mathrm{R}_y(\phi) = \begin{bmatrix} cos(\phi) & 0 & sin(\phi) & 0 \\ 0 & 1 & 0 & 0 \\ -sin(\phi) & 0 & cos(\phi) & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Let $P = (x,\ y,\ z)$ a 3D point, the rotation $R_y(\phi)(P)$ can be represented within homogeneous coordinates by:

$$\mathrm{R}_y(\phi)(\mathrm{P}) = \begin{bmatrix} cos(\phi) & 0 & sin(\phi) & 0 \\ 0 & 1 & 0 & 0 \\ -sin(\phi) & 0 & cos(\phi) & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix}$$

Exercise 9.

Within **`homogeneous.py`**, *add a function* **`rot_y_point(point, phi)`** *that takes in parameter an homegeneous vector that represents a*

`point` $(x,\ y,\ z)$ *and returns the homogeneous vector that represents the rotated* `point` *around* Y *axis by an angle* `phi` (expressed in radians).

Test the function **`rot_y_point(point, phi)`** *by displaying the point* $(4.0,\ 3.0,\ 2.0)$ *as a* ***black circle*** *and by displaying the result of its rotation around* Y *axis by an angle of* $\frac{\pi}{7}$ as a **green circle**.

Tip: *Keep in mind to convert from euclidean to homogeneous coordinates and from homogeneous to euclidean coordinates when needed.*

The rotation $R_z(\kappa)$ around Z axis by an angle κ is an application that can be represented by the matrix:

$$\mathrm{R}_z(\kappa) = \begin{bmatrix} cos(\kappa) & -sin(\kappa) & 0 & 0 \\ sin(\kappa) & cos(\kappa) & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Let $P = (x,\ y,\ z)$ a 3D point, the rotation $R_z(\kappa)(P)$ can be represented within homogeneous coordinates by:

$$\mathrm{R}_z(\kappa)(\mathrm{P}) = \begin{bmatrix} cos(\kappa) & -sin(\kappa) & 0 & 0 \\ sin(\kappa) & cos(\kappa) & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix}$$

Exercise 10.

Within **`homogeneous.py`**, *add a function* **`rot_z_point(point, kappa)`** *that takes in parameter an homegeneous vector that represents a* `point` $(x,\ y,\ z)$ *and returns the homogeneous vector that represents the rotated* `point` *around* Z *axis by an angle* `kappa` (expressed in radians).

Test the function **`rot_z_point(point, kappa)`** *by displaying the point* $(4.0,\ 3.0,\ 2.0)$ *as a* ***black circle*** *and by displaying the result of its rotation around* Z *axis by an angle of* $\frac{\pi}{8}$ as a **blue circle**.

Tip: *Keep in mind to convert from euclidean to homogeneous coordinates and from homogeneous to euclidean coordinates when needed.*

Any rotation within a 3D space can be expressed as the result of the composition of three rotations around X, Y and Z axis respectively. More formally:

$$R_{xyz}(\omega,\ \phi,\ \kappa)\ =\ R_z(\kappa) \circ R_y(\phi) \circ R_x(\omega)\ =\ \mathrm{R}_z(\kappa)\mathrm{R}_y(\phi)\mathrm{R}_x(\omega)$$

where $\mathrm{R}_z(\kappa)\mathrm{R}_y(\phi)\mathrm{R}_x(\omega)$ is the matrix product of the matrices that represents the axis rotations within homogeneous coordinates:

$$\mathrm{R}_z(\kappa)\mathrm{R}_y(\phi)\mathrm{R}_x(\omega)\ =\ \begin{bmatrix} cos(\kappa)cos(\phi) & cos(\kappa)sin(\phi)sin(\omega)\ -\ sin(\kappa)cos(\omega) \\ sin(\kappa)cos(\phi) & sin(\kappa)sin(\phi)sin(\omega)\ +\ cos(\kappa)cos(\omega) \\ -sin(\phi) & cos(\phi)sin(\omega) \\ 0 & 0 \end{bmatrix}$$

Exercise 11.

Within **`homogeneous.py`**, *add a function* **`rot_point(point, omega, phi, kappa)`** *that takes in parameter an homegeneous vector that represents a* `point` $(x,\ y,\ z)$ *and returns the homogeneous vector that represents the rotated* `point` *around the three axis* X, Y *and* Z *axis by angles* `omega`, `phi` *and* `kappa` respectively (expressed in radians).

Test the function **`rot_point(point, omega, phi, kappa)`** *by displaying the point* $(4.0,\ 3.0,\ 2.0)$ *as a* ***black circle*** *and by displaying the result of its rotation around* X, Y *and* Z *axis by angles of* $\frac{\pi}{6}$, $\frac{\pi}{7}$ *and* $\frac{\pi}{8}$ as an **orange right cross**.

Tip: *Keep in mind to convert from euclidean to homogeneous coordinates and from homogeneous to euclidean coordinates when needed.*

Scale

Scaling along X, Y and Z axis by factors σ_x, σ_y and σ_Z respectively is an application, denoted $S(\sigma_x,\ \sigma_y,\ \sigma_z)$, that can be represented by the matrix:

$$\mathrm{S}(\sigma_x,\ \sigma_y,\ \sigma_z)\ =\ \begin{bmatrix} \sigma_x & 0 & 0 & 0 \\ 0 & \sigma_y & 0 & 0 \\ 0 & 0 & \sigma_z & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Let $P = (x,\ y,\ z)$ a 3D point, the scale $S(\sigma_x,\ \sigma_y,\ \sigma_z)(P)$ can be represented within homogeneous coordinates by:

$$\mathrm{S}_x(\sigma_x,\ \sigma_y,\ \sigma_z)(\mathrm{P}) = \begin{bmatrix} \sigma_x & 0 & 0 & 0 \\ 0 & \sigma_y & 0 & 0 \\ 0 & 0 & \sigma_z & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix}$$

Exercise 12.

Within **`homogeneous.py`**, *add a function* **`scale_point(point, sx, sy, sz)`** *that takes in parameter an homegeneous vector that represents a* `point` $(x,\ y,\ z)$ *and returns the homogeneous vector that represents the scaled* `point` *along the three axis* X, Y *and* Z *axis by factors* `sx`, `sy` *and* `sz` respectively.

Test the function **`scale_point(point, sx, sy, sz)`** *by displaying the point* $(4.0,\ 3.0,\ 2.0)$ *as a* ***black circle*** *and by displaying the result of its scale along* X, Y *and* Z *axis by factors* $0,5$, $1,5$ *and* $1,0$ as a **purple right cross**.

Tip: *Keep in mind to convert from euclidean to homogeneous coordinates and from homogeneous to euclidean coordinates when needed.*

Combined homogeneous transformation

An homogeneous 3D transform is represented by a 4×4 matrix, denoted F, such as:

$$F = \begin{bmatrix} m_{00} & m_{01} & m_{02} & m_{03} \\ m_{10} & m_{11} & m_{12} & m_{13} \\ m_{20} & m_{21} & m_{22} & m_{23} \\ m_{30} & m_{31} & m_{32} & m_{33} \end{bmatrix}$$

It is possible to combine a rotation $R_{xyz}(\omega,\ \phi,\ \kappa)$, a scale $S(\sigma_x,\ \sigma_y,\ \sigma_z)$ and a translation $T(\alpha,\ \beta,\ \gamma)$ within a global transformation F such as:

$$F = \mathrm{T}(\alpha,\ \beta,\ \gamma)\mathrm{S}(\sigma_x,\ \sigma_y,\ \sigma_z)\mathrm{R}_z(\kappa)\mathrm{R}_y(\phi)\mathrm{R}_x(\omega) = \begin{bmatrix} \sigma_x cos(\kappa)cos(\phi) & \sigma_x cos \\ \sigma_y sin(\kappa)cos(\phi) & \sigma_y sin \\ -\sigma_z sin(\phi) & \\ 0 & \end{bmatrix}$$

Exercise 13.

Within **homogeneous.py**, *add a function* **create_transform(tx, ty, tz, sx, sy, sz, omega, phi, kappa)** *that takes in parameter translation parameters* `tx`, `ty` *and* `tz`, *scale parameters* `sx`, `sy` *and* `sz` *and rotation angles* `omega`, `phi` *and* `kappa` *and that return a* 4×4 `np.array` *that represent the combined homogeneous transform matrix* $F = \mathrm{T}(\alpha, \beta, \gamma)\mathrm{S}(\sigma_x, \sigma_y, \sigma_z)\mathrm{R}_z(\kappa)\mathrm{R}_y(\phi)\mathrm{R}_x(\omega)$.

Test the function by displaying the resulting matrix. The following cases have to be checked:

- *if* `tx=ty=tz=0`, *and* `sx=sy=sz=1`, *the resulting matrix is a pure rotation*
- *if* `omega=phi=kappa=0`, *and* `sx=sy=sz=1`, *the resulting matrix is a pure translation*
- *if* `omega=phi=kappa=tx=ty=tz=0`, *the matrix is a pure scale*
- *if* `sx=sy=sz=1`, *the matrix is a rigid body transformation*

Exercise 14.

Within **homogeneous.py**, *add a function* **transform(v: tuple, m)** *that takes in parameter a* `tuple v` *that represents a vector expressed within homogeneous coordinates and a* 4×4 *matrix* `m` *that represents an homogeneous transformation and that returns a* `np.array` *that represents the transformed vector expressed within homogeneous coordinates.*

The input matrix `m` *can be obtained using the* ***create_transform(tx, ty, tz, sx, sy, sz, omega, phi, kappa)*** *function.*

Test the function with various values for creating the `m` *matrix and ensure that:*

- *if* `tx=ty=tz=0`, *and* `sx=sy=sz=1`, **transform(v, m)** *is equivalent to* **rot_point(v, omega, phi, kappa)**
- *if* `omega=phi=kappa=0`, *and* `sx=sy=sz=1`, **transform(v, m)** *is equivalent to* **rot_point(v, omega, phi, kappa)**
- *if* `omega=phi=kappa=0`, *and* `tx=ty=tz=0`, **transform(v, m)** *is equivalent to* **scale_point(v, sx, sy, sz)**

TO BE CONTINUED

Vision par Ordinateur